CODEX IN VS CODE

A friendly, step-by-step guide for your phonics-games project

Learn how to ask Codex for help, give it the right files, make changes safely, and check the result. The words are simple. The guide covers the main ways to use Codex, from your first question to more advanced tools.

One important idea: Codex is a helpful coding partner. It can read, explain, and change project files. You are the boss: tell it what you want, look at the changes, and decide what to keep.

Made for a beginner using Codex in VS Code on Windows. Examples use the phonics-games project.

How to use this guide

Read it from start to finish, or jump to a chapter. Each chapter explains what Codex can do and gives you words you can paste into the chat.

1. Open Codex and ask your first question
2. Give Codex useful context
3. Ask for explanations, plans, and code changes
4. Fix bugs and review work
5. Check and test the result
6. Set project preferences
7. Use Git save points
8. Explore skills, MCP, agents, and cloud
9. Follow a complete game example
10. Use prompt helpers and solve common problems

New words: A file is a saved page of code or information. A project is a folder with related files. A prompt is the request you type. A diff shows what changed.

1. What is Codex?

Codex is an AI helper for coding. In VS Code, it sits beside your project. Ask it questions, ask it to find things, or ask it to make a change. It can look at files you give it, edit files, and sometimes run commands that you approve.

Things Codex can help with

- Explain: Tell you what a file or piece of code does in beginner-friendly words.
- Find: Search for a button, word list, function, page, or error across the project.
- Plan: List the steps and files needed before making a change.
- Build: Add a feature, page, game, or small improvement.
- Fix: Trace a problem and suggest or make a repair.
- Improve: Clean up code while keeping the same behavior, if you ask.
- Review: Look for mistakes and explain what to check.
- Work with data: Help read or update HTML, CSS, JavaScript, JSON, and text.
- Help you learn: Explain why a change works and define coding words.

Codex can misunderstand you or make mistakes. Treat its answer like a first draft: read it, try it, and decide whether it is right.

2. Open Codex in VS Code

Open the phonics-games folder in VS Code. Click the Codex icon to open its sidebar. If you cannot see it, press Ctrl+Shift+P, type Codex: Open Codex Sidebar, and press Enter. Sign in if Codex asks.

Start with a question

Ask Codex to explain one file before asking it to change the whole project. This helps you learn and check that Codex understands the code.

```
Explain templates/P-pic-word.html in simple words.
Do not edit any files. Tell me how the game starts,
checks an answer, and moves to the next word.
```

If the answer is confusing, ask it to explain one part more simply. You can learn in small steps.

3. Give Codex useful context

Context means the information Codex needs to do your task well. Point it to the right parts so it does not have to guess.

- Open the file you are talking about in the editor.
- Select the lines you mean, then ask about those lines.
- Name the file or folder, for example `templates/P-pic-word.html`.
- Paste the error message or say what you clicked and what happened.
- For a change involving many files, ask Codex to find the connected files first.
- If your extension lets you attach files or mention file names, include the useful ones.

```
Find where the Challenge 1 tile gets its game-page link.
Show me the file and code that controls it. Do not edit.
```

This project has many standalone activity pages. A game may get its words from a local list or JSON lesson file, while another page links to it. Ask Codex to trace those connections when a change touches more than one part.

4. Write prompts that get good results

A prompt is what you ask Codex. A good prompt says what you want, where the work belongs, what must stay the same, and how you will know it is finished.

A simple prompt recipe

```
GOAL: What should change?
WHERE: Which file, page, or feature?
RULES: What must it do? What should stay the same?
DONE WHEN: What can I see or try to know it worked?
```

Example: add a replay button

```
Goal: Add a replay button to the picture-word game.
Where: templates/P-pic-word.html
Rules: Replay starts a fresh round. Reset the score and
progress. Keep the current word list and game screens.
Done when: I can finish a round, choose Replay, and
start again with a clean score.
```

Say if you want ideas or edits

If you are still thinking, ask for ideas and say “do not edit.” If you want it built, say “make the change.” Then Codex knows whether to talk or change files.

```
I am thinking about adding a sound button. Give me three
simple ideas and say which files each may change. Do not edit.
```

You do not need programmer words. Describe what a child should be able to do. Codex can work out which code may need changing.

5. Learn before making a big change

For a larger request, start with a small investigation. Ask Codex to read the related code and explain the plan. Check the plan, then tell it to build the change.

```
I want children to choose an easier or harder word level.
First, find how this game stores and chooses words. Tell me
which files may need to change and suggest a small plan.
Do not edit yet.
```

After you read the answer, you might say:

```
That plan looks good. Make the change now. Keep it small,
keep the current game working, and tell me how to check both
levels in the browser.
```

For a tiny change, like fixing a spelling mistake, you can ask Codex to do it right away. Ask for a plan when the task is hard to describe or touches several files.

6. Use Codex for common coding jobs

Understand code

Explain this function like I am new to JavaScript. What goes in, what changes, and what comes out?

Find a feature

Find the code that shows the Correct Answer screen. Explain what makes it appear.

Make a feature

Add a “Hear the word” button. Use the existing word data. Make it large enough to tap on a phone. Keep the other game rules.

Fix a bug

When I tap Next Word, the same word appears again. Trace how it picks the next word, fix the cause, and explain the change.

Improve the screen

Make the answer buttons easier for a young child to tap on a phone. Keep this page’s colors and layout.

Work with learning words

Suggest 12 easy Grade 1 words with “sh”. Avoid harder sound patterns. Show them in this activity’s data format. Do not add them yet.

Write instructions

Write short “How to Play” instructions for a 7-year-old. Use short sentences and numbered steps. Do not change game code.

You can ask about HTML (page parts), CSS (how a page looks), JavaScript (how buttons and rules work), or JSON (saved lists of information). Say which part you want explained.

7. Review edits before keeping them

When Codex edits files, it should show a summary and a change view called a diff. A diff marks lines that were added, changed, or removed.

- Read the summary. Does it match what you asked for?
- Look at every changed file. Did Codex edit the right place?
- Read the changed lines. Do they follow your rules?
- Look for extra changes that are not needed.
- Ask Codex to explain a line or fix it if you are unsure.
- Keep the edits you want. Undo or revert edits you do not want.

```
Please explain the diff in simple words. Point out any
change that might affect another activity.
```

You can ask Codex to review without editing:

```
Review the changes for bugs. Look at the Replay button
and score reset. Do not edit. Tell me what could go wrong.
```

Depending on your setup, `/review` may be available for starting a review. The extension can be set to show the review in the current chat or in a separate chat.

8. Check that the work behaves correctly

Code can look neat and still have a bug. Try the activity the way a child or teacher will use it. Ask Codex what to check and what it actually tested.

Things to check in a game

- Can you start, answer, finish, and replay?
- Does the score or progress update at the right time?
- What happens for a wrong answer, an empty list, or the last word?
- Can you use it on a small phone screen?
- Can you use a keyboard if that matters for this game?
- Do the words fit the lesson you want to teach?

```
Tell me how to check this change in my browser. List the
steps and what I should see. Tell me what you checked and
what I still need to check myself.
```

This project’s preview

This project has a VS Code Live Server setting on port 5501. If Live Server is installed and running, it can show local HTML pages in a browser. Open the changed page through the project so its nearby pictures and files can load.

> Only say a feature works after checking it in the real page or with a suitable check. If Codex did not open the page or run a check, ask it to say so plainly.

9. Keep instructions in the right place

AGENTS.md: rules for this project

Your project already has an `AGENTS.md` at its top level. It gives Codex lasting instructions for this project: make learning activities for young children, keep them playful, support phones and touch, follow each activity's style, reuse learning data, and use Ghanaian examples when they fit.

Add a rule there only if you want it to guide future tasks in this project. Keep it short. For example:

```
- Use plain English in student instructions.
- Keep game controls large and easy to tap.
- Ask before adding a new library or service.
```

Your prompt: rules for this task

Put a one-time request in your message. For example: "For this change only, use a blue background." Then that special choice will not become a rule for every task.

Codex and VS Code settings

Use the gear in the Codex sidebar and choose Codex Settings for agent choices such as model, reasoning, permissions, personal preferences, and connected tools. Open VS Code Settings and search for `@ext:openai.chatgpt` or `Codex` to find editor choices. What you see can depend on your version and account.

Your VS Code setting `liveServer.settings.port` is 5501. It sets the preview server, not how Codex thinks.

10. Use Git as a save point

Git keeps a history of file changes. A Git checkpoint can help you go back if a change causes trouble. Before a big task, check the Source Control panel and save work you want to keep. After Codex edits, look at the diff before saving another checkpoint.

```
Before editing, tell me if these files already have changes.
Do not remove or replace work that was already there.
```

Ask Codex to explain Git changes or suggest a commit message. Be careful with actions that erase history or replace work on the internet. Read what the command does before allowing it.

11. Permissions: decide what Codex may do

Depending on its settings, Codex may read files, edit files, run commands, or use the internet. When it asks to do something, check what it wants to do and where it will happen.

- Reading a file is different from changing it.
- Running a small local command is different from installing software.
- A command that deletes files or changes many files needs a careful look.
- Using the internet can send information outside your computer.
- Give only the access needed. If you do not understand a command, ask what it does before allowing it.

```
Before you run that command, explain what it does, which
files or services it changes, and whether it needs the internet.
```

A sandbox is a fence around what Codex can reach. Codex settings can include permission and sandbox choices. Use only the access needed for your task, and follow any rules set by your school or workplace.

12. Work with the editor while you chat

Open the right page or select the lines you mean. Ask a focused question. Keep asking follow-up questions in the same chat while working on the task.

- “What does this selected code do?”
- “Change only this part so the button shows when it is disabled.”
- “I tried your fix. The error is still there: [paste error]. Continue from the current code.”
- “Stop and tell me what has changed so far.”

If Codex missed something, say so clearly: “Please keep the sound button. Your last change removed it.”

13. Skills, MCP, and other helpers

Skills: saved how-to cards

A skill is a saved set of steps for a kind of job. It might teach Codex how your team wants it to review code or make a lesson page. You can ask for an installed skill by name, or let Codex pick one that fits when supported. Only use skills you trust; some may include small programs.

MCP: a connector to another tool

MCP is a way to connect Codex to extra tools or information, such as a help library or another service. Add a connection only when you need it, and check what it can read or do. An OpenAI Docs MCP connection can search official OpenAI help pages; it does not code your project for you.

Other agents and cloud work

Some setups can send longer work to a cloud computer or use another helper agent. This can help with a big, separate job. You still need to check the final changes. For one small game edit, working right in VS Code is usually simpler.

14. Full example: make a phonics game

Imagine you want a game that helps Grade 1 pupils hear the “sh” sound.

Step 1: Look around

```
I want a Grade 1 game for the “sh” sound. First, look at
similar activities and the word/lesson data. Explain which
page is closest and what we can reuse. Do not edit.
```

Step 2: Describe the game

```
Make a simple picture-and-word game using the closest
existing activity as a guide. A pupil hears a word, chooses
one of three answers, gets friendly feedback, and plays again.
Use easy “sh” words from project data when possible. Make it
work on phones. Keep the current page style.
```

Step 3: Say when it is done

```
Done means: the game starts; the audio button plays a word;
the pupil picks an answer; feedback appears; the next word
loads; the final screen offers replay. Tell me which files
changed and how to preview it.
```

Step 4: Review and try it

Read the summary and diff. Check that Codex used the right lesson words, linked the new page if needed, and did not change another activity by mistake. Open the game in the browser. Try right and wrong answers, sound, the last question, replay, and a narrow screen. If something fails, tell Codex what you clicked and saw.

15. Prompts you can keep nearby

Explain

Explain [file or code] in simple words. Do not edit.

Find

Find where [button/word/rule] is controlled. Name the files and explain how they connect. Do not edit.

Plan

I want [goal]. Look at the project, list files that may need changes, and suggest steps. Do not edit yet.

Build

Make [goal] in [file/feature]. Keep [important parts]. It is done when [visible result]. Tell me what changed.

Fix

I expected [A], but [B] happened after [steps]. Find the cause, fix it, and explain how I can check.

Review

Look for bugs in the current changes. Do not edit. List problems and how to check them.

Teach

Explain why this code works. Define new words and show one small example.

Make smaller

This change is too big. Keep what is needed and reduce the edits. Explain what you remove.

Check

Tell me what checks you ran, what passed, what failed, and what I should still try.

16. Common problems and what to try

Codex changed the wrong file

Say: “Undo only the unrelated file change. Keep the requested change. Tell me which files will stay changed.” Then inspect the diff.

The answer is too hard

Say: “Explain in three short steps, using beginner words.” Or ask about one function or file.

Codex does not know which game

Give the file name, open the page, or select the code. Ask it to find the matching activity and explain why.

The page still fails

Give the exact steps, paste the browser error, and say what you expected. Ask Codex to investigate and report what it checked.

The task takes too long

Ask for a short status and plan. Break the work into smaller parts, such as one screen at a time.

17. A safe, useful daily routine

- Before: Open the project and look for existing edits.
- Ask: Write the goal and what should stay the same.
- Guide: Point to files, selected code, or an error.
- Work: Ask for a plan first if the task is large.
- Review: Read the summary and inspect the diff.
- Check: Try the activity or ask for the right check.
- Keep: Save a Git checkpoint for work you want.
- Learn: Ask Codex to explain anything you do not understand.

Remember: You do not need to know coding words. Describe what you want in everyday words. Share the right files, ask for small steps, and check the result yourself.

Official guides for later

These official OpenAI pages explain the Codex extension, project instructions, settings, permissions, skills, and documentation connections.

Codex IDE extension
https://learn.chatgpt.com/docs/codex/ide

AGENTS.md project instructions
https://learn.chatgpt.com/docs/agent-configuration/agents-md

Developer settings
https://learn.chatgpt.com/docs/developer-settings

Permission modes
https://learn.chatgpt.com/docs/permission-modes

Skills
https://learn.chatgpt.com/docs/build-skills

OpenAI Docs MCP
https://learn.chatgpt.com/learn/docs-mcp

Codex changes over time. Your installed version or account may show different buttons. Use the settings and help links in your Codex panel if something looks different.